Tooth Diagram Generation: Temperley No-Overhang Stacked Partitions

Summary of `generate_tooth.py`

(a) Theory

Teeth and the no-overhang condition

The paper defines a *tooth of base* r as a finite stack of horizontal rows of unit cubes satisfying three conditions:

1. The bottom row (the *base*) has exactly r cubes.
2. The row lengths $\lambda_1 = r \geq \lambda_2 \geq \cdots \geq \lambda_h \geq 1$ are weakly decreasing upward.
3. No row overhangs the row immediately beneath it.

Formally, if $x_i \in \mathbb{Z}$ denotes the left-hand offset of row i, the no-overhang condition is

$$0 \leq x_{i+1} - x_i \leq \lambda_i - \lambda_{i+1}, \qquad 1 \leq i \leq h-1.$$

The total number of cubes is $|T| = \sum_{i=1}^{h} \lambda_i$, and the generating function $h_r(q) = \sum_T q^{|T|}$ sums over all teeth of base r.

Canonical decomposition

The paper encodes each tooth via a canonical bijection (Proposition 2) as a triple (A, B, c) of independent non-negative integer sequences. The decomposition parameters for each consecutive pair of rows i and $i+1$ are:

$$a_i = x_{i+1} - x_i \quad (\text{left shift}),$$

$$b_i = (\lambda_i - \lambda_{i+1}) - a_i \quad (\text{right inset}),$$

with $c = \lambda_h$ (the top row length). These satisfy $a_i, b_i \geq 0$ and $\sum(a_i + b_i) + c = r$. The decomposition parameters are the combinatorial quantities from which the height-refined product formula (Theorem 2) and the Gaussian-binomial expansion (Theorem 3) are derived.

(b) What the script does

The script generates an SVG diagram of a tooth from a user-supplied specification of row lengths and left offsets.

It performs four main tasks:

1. **Input parsing**: reads the tooth specification as a sequence of `(length, offset)` pairs, one per row, with the base row given first.

2. **Validation**: checks that the specification defines a valid tooth, enforcing both the weakly-decreasing length condition and the no-overhang condition for every consecutive pair of rows.
3. **SVG generation**: renders the tooth as a grid of unit-cube cells, drawn bottom-to-top (base row at the bottom, in the standard orientation used in the paper).
4. **Decomposition annotation** (optional): labels the canonical decomposition parameters a_i, b_i, and c directly on the diagram, with bracket lines between consecutive rows and a summary line below the diagram.

(c) How the script does it

Input format

Rows are specified on the command line as a space-separated list of `length,offset` tokens, base row first. For example, the tooth of base $r = 7$, height $h = 3$ with rows $(\lambda_1, \lambda_2, \lambda_3) = (7, 4, 2)$ and offsets $(x_1, x_2, x_3) = (0, 2, 3)$ is given as:

```
--rows "7,0 4,2 2,3"
```

Validation

`validate_tooth(rows)` enforces three conditions in order:

1. All row lengths are positive.
2. Row lengths are weakly decreasing: $\lambda_{i+1} \leq \lambda_i$ for all i.
3. The no-overhang condition $0 \leq x_{i+1} - x_i \leq \lambda_i - \lambda_{i+1}$ holds for every consecutive pair.

A `ValueError` with a descriptive message is raised on any violation. The base row offset is additionally required to be zero (i.e. $x_1 = 0$), fixing the horizontal position of the tooth.

SVG layout

The SVG canvas width is determined by the maximum rightward extent $\max_i(x_i + \lambda_i)$ across all rows, and the height by the number of rows h. Each unit cube is drawn as a rectangle of side `box_size` pixels with `spacing` pixels between adjacent cells.

Rows are drawn bottom-to-top: row $i = 1$ (the base) occupies the lowest SVG coordinates, and row $i = h$ (the top row) the highest. This matches the standard orientation of tooth diagrams in the paper.

Decomposition annotation

`compute_decomposition(lambdas, offsets)` computes:

$$a_i = x_{i+1} - x_i, \qquad b_i = (\lambda_i - \lambda_{i+1}) - a_i, \qquad c = \lambda_h.$$

When `--annotate` is specified, the script draws:

- A horizontal bracket line and an a_i label above the left-shift region between rows i and $i+1$ (omitted when $a_i = 0$).
- A horizontal bracket line and a b_i label above the right-inset region between rows i and $i+1$ (omitted when $b_i = 0$).
- A c label above the top row.
- A summary line below the diagram of the form `r=7, h=3: c=2 (a=2,b=1) (a=1,b=0)`.

The SVG canvas height is extended automatically to accommodate the summary line when `--annotate` is used.

Gradient support

If `--gradient` is supplied with two comma-separated colours, a diagonal linear gradient is defined in the SVG `<defs>` block and applied as the cell fill. This overrides `--fill`.

(d) Output produced

The script writes a single SVG file (or prints to stdout if no `-o` path is given). The SVG contains one `<rect>` element per unit cube, optionally supplemented by `<text>` and `<line>` elements for the decomposition annotations.

Verification of the paper's example

The tooth of base $r = 7$, height $h = 3$ described in Figure 2 of the paper (rows $\lambda = (7, 4, 2)$, offsets $(0, 2, 3)$) is reproduced by:

```
python generate_tooth.py --rows "7,0 4,2 2,3" --annotate -o tooth.svg
```

This produces a diagram with 13 cells ($7 + 4 + 2 = 13$), annotated with $a_1 = 2$, $b_1 = 1$, $a_2 = 1$, $b_2 = 0$, $c = 2$, consistent with the decomposition $r = c + (a_1 + b_1) + (a_2 + b_2) = 2 + 3 + 2 = 7$ shown in the paper.

(e) Command-line options

Option	Default	Description
`--rows` / `-r`	*(required)*	Row specs as `"length,offset"` pairs, base row first
`-o` / `--output`	stdout	Output SVG file path
`--box-size`	`40`	Cell side length in pixels
`--spacing`	`2`	Gap between adjacent cells in pixels
`--fill`	`#4f46e5`	Cell fill colour
`--stroke`	`#312e81`	Cell border colour

Option	Default	Description
`--stroke-width`	`2.0`	Cell border width in pixels
`--rounded`	`4`	Corner radius for rounded cells
`--padding`	`20`	Margin around the diagram in pixels
`--gradient`	—	Two colours for a diagonal gradient fill (overrides `--fill`)
`--font-size`	`12`	Font size for cell annotations
`--font-family`	`sans-serif`	Font family for all text
`--annotate`	off	Label a_i, b_i, c decomposition parameters
`--annot-color`	`#e11d48`	Colour for decomposition annotation text and lines
`--annot-font-size`	`11`	Font size for decomposition annotations

(f) Relation to the companion scripts

`generate_tooth.py` is a visualisation tool rather than a verification or computation script. It does not compute generating functions, verify recurrences, or enumerate teeth; those tasks are handled by the companion scripts `generate_tooth_sequences.py`, `verify_recurrence.py`, `verify_theorem1.py`, `verify_theorem3.py`, and `verify_conjecture_and_tables.py`.

Its primary role is to produce publication-quality SVG diagrams that illustrate the combinatorial objects studied in those scripts and in the paper. The `--annotate` flag in particular makes explicit the canonical decomposition that underlies Theorems 2 and 3, providing a visual counterpart to the algebraic identities verified elsewhere.